

모델배포 개발된 LLM 연동 웹 애플리케이션

산출물 단계	모델배포
평가 산출물	LLM 연동 웹 애플리케이션
제출 일자	2026-02-06
깃허브 경로	SKN 19기 FINAL 5팀 github
작성 팀원	박도연

1. 프로젝트 개요

- 프로젝트명: 한국 소비자 분쟁 조정 상담을 위한 멀티 에이전트 챗봇 시스템(똑소리)
- 작성일: 2026-02-06
- 작성자: 박도연

2. 시스템 개요

- 목표: 본 프로젝트는 복잡하고 전문적인 한국의 소비자 분쟁 관련 문의에 대해 정확하고 신뢰도 높은 답변을 제공하는 MAS(Multi-Agent System) 챗봇을 개발하는 것을 목표로 합니다.
- 기능:
 - PostgreSQL + pgvector 기반 벡터 데이터베이스 구축 및 유사도 검색
 - RAG 구조를 활용한 문서 검색 기반 LLM 응답 생성
 - 멀티 에이전트 기반 질의 처리 파이프라인 구성
 - FastAPI 기반 백엔드 API 서버 구현
 - LLM 응답 검증 로직을 통한 환각(Hallucination) 최소화
 - 환경 변수 기반 API 키 관리 및 예외 처리
 - 실시간 응답 스트리밍 지원

3. 시스템 아키텍처

- 인프라: AWS EC2, RDS, Nginx, Docker, Github Actions
- 백엔드:
 - 언어: Python
 - 프레임워크: LangGraph, FastAPI
 - LLM 연동: OpenAI API (gpt-4o, gpt-4o-mini)
 - 벡터 데이터베이스: PostgreSQL(pgvector)
 - 데이터베이스: PostgreSQL
 - 기술 스택:
 - 인증/보안: OAuth 2.0 (Google, Naver), JWT, Rate Limiting(slowapi), CORS 제어
 - Github Actions 활용 CI/CD 구축
- 프론트엔드:
 - 프레임워크: React, Vite, TypeScript
 - 상태/데이터: TanStack React Query, Zustand
 - 스타일/UI: Tailwind CSS
 - 통신 방식: fetch 기반 비동기 요청 + SSE 스트리밍 응답 처리

4. LLM 연동 및 벡터 데이터베이스 구현

4.1 벡터 데이터베이스와 LLM 연동

- 연동 목적: 사용자의 소비자 분쟁 질의에 대해 문서 기반(RAG)으로 답변하여, 근거 없는 단정/환각을 줄이고 신뢰도를 높입니다. 검색된 근거(법령/해결기준/사례)를 답변과 함께 제공하여 사용자가 출처를 확인할 수 있도록 했습니다.
- 연동 방식(핵심 흐름):
 1. 질의 분석(Query Analysis): 질의 유형 분류(RAG 필요 vs 불필요), 키워드 추출, 확장 쿼리 생성
 2. 문서 검색(Retrieval): PostgreSQL(pgvector)에서 Dense 검색 + 키워드 검색을 수행, RRF로 결합
 3. 답변 생성(Answer Generation): LLM(OpenAI GPT-4o)에 컨텍스트와 함께 프롬프트를 전달해 답변 생성
 4. 법률 검토(Law Review): 답변의 단정 표현/환각 가능성/추가 질문 필요

여부를 검토하여 최종 응답 확정

5. 스트리밍 반환: 프론트로 스트리밍 형태로 응답 전송

- 프롬프트:

- 근거 우선: 제공된 컨텍스트에서만 답변하도록 제한하고, 부족하면 “추가 질문” 또는 “알 수 없음”을 반환
- 출처 표기: 답변에 근거 문서의 출처(문서 유형/인용 구간 등)를 함께 제시
- 안전장치: 법률/분쟁 관련 고위험 내용은 단정 대신 조건/가능성 중심으로 표현하고 필요 시 추가확인 유도

예시 코드: 검색된 컨텍스트와 답변 생성

```
result = generator.generate_structured_answer(
    query=query,
    agency_info=dict(agency_info),
    counsels=list(retrieval.get("counsels", [])),
    laws=list(retrieval.get("laws", [])),
    criteria=list(retrieval.get("criteria", [])),
    retry_supplement=retry_supplement,
    onboarding=dict(onboarding) if onboarding else None,
    system_prompt=system_prompt,
    user_prompt=user_prompt,
)

response = self.client.chat.completions.create(
    model=self.model,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    temperature=0.3,
)
```

5. 예외 처리 및 보안

5.1 예외 처리

- 예상치 못한 상황에 대한 예외 처리: 외부 **API** 호출 실패, **DB** 연결 실패, 검색 결과 없음 등 케이스를 구분하여 사용자에게 적절한 메시지를 제공합니다.
- 검색 결과가 충분하지 않으면:
 1. 추가 질문을 통해 사건 사실관계/거래 유형/시점 등을 보완 요청하거나
 2. “제공된 정보로 단정하기 어렵다”는 안내와 함께 일반적인 참고 정보를 제한적으로 제공
- 서버 오류 발생 시: **/health** 기반 헬스체크와 로그를 통해 장애 원인을 추적할 수 있도록 구성합니다.

예외 처리 코드 예시: 채팅 응답 생성 파이프라인에서 발생한 미처리 예외를 포착해, 서버 로그/요청 로그를 남기고 **500** 오류로 응답하는 전역 예외 처리

except Exception as e:

```
logger.exception("[chat] Unhandled error")
```

```
rag_logger.log_response(
```

```
    entry=log_entry,
```

```
    chunks_used=0,
```

```
    sources_count=0,
```

```
    status="error",
```

```
    error_message=str(e),
```

```
)
```

```
rag_logger.finalize(log_entry, start_time)
```

```
rag_logger.save(log_entry)
```

```
raise HTTPException(status_code=500, detail=f"답변 생성 중 오류 발생: {str(e)}")
```

5.2 보안 관리

- 민감 정보 보호 원칙:
 - **OPENAI_API_KEY**, **DB** 접속 정보, **JWT/OAuth** 시크릿 등 민감정보는 코드에 하드코딩 하지 않고 환경변수 기반으로 관리
 - 개발 환경에서는 **.env**를 사용하고, 저장소에는 **.env.example** 등 템플릿만 유지
 - 배포 환경(**AWS**)에서는 **Docker** 기반 실행 시 환경변수로 주입하며, 실제 시크릿 값은 **AWS Secrets Manager**로 관리
 - 백엔드는 **USE_AWS_SECRETS=true** 설정 시 **AWS Secrets Manager**에서 시크릿을 로드해 애플리케이션 환경변수로 주입하도록 구성

6. 코드 모듈화 및 주석

6.1 모듈화

- 모듈 분리 목적:
 - 기능별 책임을 분리하여 유지보수성을 높이고, 데이터 도메인(법령/상담/사례)별 검색 로직을 독립적으로 개선 가능하도록 구성
- 주요 모듈(역할 기준):
 - Query Analysis: 질의 의도 분류, 키워드/필터 추출
 - Retrieval: 벡터 검색/키워드 검색/하이브리드 결합(RRF) 및 컨텍스트 구성
 - Answer Generation: LLM 호출 및 답변 포매팅(근거 포함)
 - Review: 법률 검토 및 단정 표현 완화/추가 질문 트리거
 - API Layer(FastAPI): 라우팅, 요청/응답 스키마, 스트리밍 전송

6.2 주석

주석 예시:

```
def output_guardrail_node(state: Dict[str, Any]) -> Dict[str, Any]:
```

```
    # === PR-6: draft_answer를 final_answer로 복사 ===
```

```
    draft_answer = state.get("draft_answer", "")
```

```
    final_answer = state.get("final_answer", "")
```

```
    # 우선순위: final_answer > draft_answer
```

```
    if not final_answer:
```

```
        final_answer = draft_answer
```

```
    # 안전 체크: 둘 다 비어있으면 에러 로그 + fallback 메시지
```

```
    if not final_answer:
```

```
        logger.error(
```

```
            "[OutputGuardrail] CRITICAL BUG: Both draft_answer and final_answer are empty! "
```

```
            f"State keys: {list(state.keys())}"
```

```
        )
```

Fallback 메시지로 사용자 경험 개선

`final_answer` = "죄송합니다. 일시적인 오류로 답변을 생성하지 못했습니다. 잠시 후 다시 시도해주세요."